1. What is Data Persistence?

Data persistence is the ability to **store data in permanent storage** so that it is not lost when an application, computer, or server is restarted or shut down.



If you enter data, close the application, and open it again, the data should still be available.
That is data persistence.

2. Example 1: Student Database

A college application stores student information in a database.

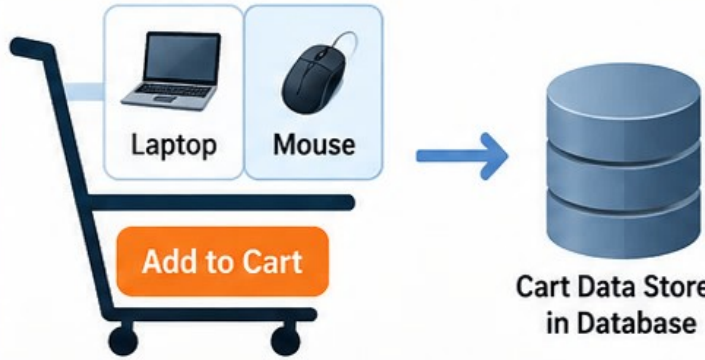
Student ID	Name	Branch	Marks
101	Ravi	CSE	85
102	Sneha	ECE	78
103	Arjun	IT	92



Even if the computer is switched off or restarted, the student information remains available.
The database provides data persistence.

3. Example 2: Online Shopping

You add products to your shopping cart.



You close the website and come back later.
The cart still contains the same products.

The shopping system has persisted the cart data in a database associated with your account.

4. Example 3: Mobile Banking

ou have an account balance and make a transaction.



Even if the banking application is closed or the server is restarted, the updated balance remains ₹ 45,000.
The transaction and data are stored permanently in a database.

5. Persistent vs Non-Persistent Storage

Type	Where data is stored	What happens after shutdown?
Non-persistent	RAM (Main Memory)	Data is normally lost
Persistent	Database / Disk / SSD / Cloud Storage	Data remains available

In NoSQL databases (e.g., MongoDB, Cassandra, Redis depending on configuration), data is also written to persistent storage so it can be recovered after a server restart.

. What is Impedance Mismatch?

mpedance mismatch is the difficulty that occurs when he way data is represented in an **application** rogramming language) is different from the way the ame data is represented in a **database.**

The application thinks about data in one way, while the database stores it in another way. Converting between the two representations creates extra work.

xample: Student Data

pplication bject-oriented model)

Java

```
ass Student {
 int studentId;
 String name;
 String branch;
 List<String> subjects;
 Address address;

ass Address {
 String city;
 String pincode;
```

e **Student** object with sted objects and collections

Conversion (ORM)

Relational Database (Table model)

STUDENT		
student_id	name	branch
101	Ravi	CSE

ADDRESS		
student_id	city	pincode
101	Guntur	522006

SUBJECT		
student_id	subject	marks
101	NoSQL	85
101	DBMS	78
101	ML	92

Data is split across multiple tables with relationships

Why is it a mismatch?

- Application uses objects, nested structures, lists, etc.
- Database uses tables, rows, columns, and relationships.
- Extra code is needed to convert between the two.
- This conversion is called Object-Relational Mapping (ORM).
- This difference is known as **impedance mismatch.**

2. Example 1: E-Commerce

An e-commerce application represents a customer with multiple orders and products as a single object, but a relational database stores the same data in multiple tables.

Application View (Object Model)

Customer
ID: 101
Name: Ravi
Email: ravi@gmail.com

Order 1001
- Laptop
- Mouse

Order 1002
- Keyboard

In the application, a customer has many orders, and each order contains multiple products.

Relational Database View (Table Model)

CUSTOMER		
ID	NAME	EMAIL
101	Ravi	ravi@gmail.com

ORDERS	
ORDER_ID	CUSTOMER_ID
1001	101
1002	101

ORDER_ITEMS	
ORDER_ID	PRODUCT
1001	Laptop
1001	Mouse
1002	Keyboard

The same customer data is stored in three tables.

Impedance Mismatch Here

- The application sees a single nested object.
- The database stores the data in separate tables.
- To retrieve the complete customer, the application must perform joins and reconstruct the object.
- This creates extra code and complexity.

How NoSQL Helps

A document database can store the customer with orders and products in a single document, which is closer to the application's object structure.

3. Two More Real-Time Examples

Example 2: Social Media Application

Application View (User Object)

User
ID: 101
Name: Ravi
Age: 21

Location
City: Guntur
State: AP

Skills
- Python
- Java
- MongoDB

Friends
- Arun
- Priya
- Sita

Mapping between different models

Relational Database View

USER		
user_id	name	age
101	Ravi	21

LOCATION		
user_id	city	state
101	Guntur	AP

SKILLS	
user_id	skill
101	Python
101	Java
101	MongoDB

FRIENDS	
user_id	friend_id
101	201
101	202
101	203

Mismatch
- Application uses a nested user object with lists.
- Database stores the data in multiple related tables.
- The application must join and reconstruct the object.

Example 3: Banking / Financial Application

Application View (Account Object)

Account
No: 10001
Balance: ₹50,000

Customer
Name: Ravi
Contact: 98765...

Transactions
- Deposit ₹10,000
- Withdraw ₹5,000
- Transfer ₹2,000

Mapping between different models

Relational Database View

ACCOUNT		
account_no	customer_id	balanc
10001	C101	50000

CUSTOMER	
customer_id	name
C101	Ravi

TRANSACTION			
trans_id	account_no	type	amo
T001	10001	DEPOSIT	100
T002	10001	WITHDRAW	50
T003	10001	TRANSFER	20

Mismatch
- Application uses one account object with a list of transactions.
- Database stores the data in multiple tables.
- The application must retrieve, join, and reconstruct the object.
- This adds development complexity.

Key Takeaway

Impedance mismatch occurs when the application and database represent the sa data in different ways, requiring conversion, mapping, or transformation.

edance Mismatch: Meaning

nce mismatch refers to the difficulty that arises when the data model used in a relational dat
ent from the data model used in a programming language (in-memory data structures).
ucture of data in a relational database is based on tables (rows and columns), while in a prog
typically represented using objects and related structures. Because of this difference, data ca
rectly and must be converted between the two models.

ational Database (Table Model)

ored in tables with rows and columns.

Student

_ID	Name	Department	Marks
	Ravi	CSE	85
	Sita	ECE	91
	Kumar	ME	78

in a flat, tabular structure.
nships are represented using keys
rimary key, foreign key).

Impedance Mismatch

Different data models require conversion between the two representations.

In-Memory Data Structures (Object

Data is represented using objects, classes and

```
class Student {
   int studentId;
   String name;
   String department;
   int marks;
}
```

Student

studentId:
name: "Rav
departmen
marks: 85

- Data is stored as objects with attributes.
- Relationships are represented using object r
(e.g., a Student object may contain a Depart

Is Called "Impedance Mismatch"?

erm "impedance mismatch" comes from electrical engineering, where it means a difference in impedance
prevents efficient transfer of signals.
arly, in software systems, the difference between the relational model and the object model makes it difficu
nsfer data directly, requiring an extra layer to convert between the two.
conversion adds complexity and can affect performance.

Manifests

om tables must be mapped to objects (e.g., a row → an object).
must be converted back to tables for storage (e.g., an object →
e rows).
x object structures (with nested objects or collections) must be
o multiple tables.
nships represented by object references must be translated
eign keys.
bers often need to use an Object-Relational Mapping (ORM)
ork (e.g., Hibernate, JPA) to handle this mapping.

Example: One-to-Many Relationsh

A Department has many Students.

Relational Tables

Department

Dept_ID	Dept_Name
D01	CSE
D02	ECE

Student

Student_ID	Name	Dept_ID
101	Ravi	D01
102	Sita	D01
103	Kumar	D02

Object Mo

Departme
String deptId;
String deptNam
List<Student> s

1
*

Studen
int studentId;
String name;
Department dep

Conclusion

Impedance mismatch exists because the relational and object models are fundamentally different.
equires an additional mapping layer, which can add complexity and affect performance, especially in large applicat

1. Impedance Mismatch – Concept

npedance mismatch refers to the difference between how an application represents ata in memory (using objects, classes and references) and how a relational database tores data (using tables, rows and columns).

Application (Object Model)

- **Order Object**
  - Customer (object)
  - List of Line Items (collection of objects)
  - Payment (object)

Impedance Mismatch

Different data models require conversion between the two representations.

Relational Database (Table Model)

- CUSTOMER table
- ORDERS table
- ORDER_ITEM table
- PAYMENT table

The same information is organized differently in the application and in the database, so data must be mapped between the two models.

2. Application View – Order Object in Memory

In the application, an order can be represented as a single object containing customer details, multiple line items and payment information.

Order Object

orderId: 1001
orderDate: 07-09-2026

Customer (object)

customerId: 101
name: "Ravi"
email: "ravi@example.com"
phone: "9876543210"

Line Items (list of objects)

Item 1	Item 2
product: "Laptop"	product: "Mouse"
quantity: 1	quantity: 2
price: 50000	price: 3000

Payment (object)

paymentId: 901
method: "Credit Card"
amount: 56000
status: "Successfull"

Key Points (Application View)

- Represented as one connected object structure.
- Customer is an embedded object
- Line items are a collection (list) of objects.
- Payment is another object.
- Easy to work with in the applicati using object references.

3. Relational Database Representation

a relational database, the order, customer, line items and payment information are stored in separate bles and linked using primary keys and foreign keys.

CUSTOMER

tomer_ID	Name	Email	Phone
101	Ravi	ravi@example.com	9876543210
102	Sita	sita@example.com	9123456789
...	...	...	...

1 places M

ORDERS

Order_ID	Customer_ID	Order_Date
1001	101	07-09-2026
1002	102	08-09-2026
...	...	...

1 contains M

1 has 1

ORDER_ITEM

m_ID	Order_ID	Product_ID	Quantity	Price
1	1001	201	1	50000
2	1001	202	2	3000
...	...	...	...	...

PAYMENT

Payment_ID	Order_ID	Method	Amount	Status
901	1001	Credit Card	56000	Successful
902	1002	UPI	6000	Successful
...	...	...	...	...

Primary keys and foreign keys are used to maintain the relationships between tables.

4. Mapping, Retrieval and Key Takeaways

Storing the Order

1. Store customer details in CUSTOMER table (if not already present).
2. Store order information in ORDERS table (with customer_id as foreign key).
3. Store each line item as a separate row in ORDER_ITEM table (with order_id as foreign key).
4. Store payment information in PAYMENT table (with order_id as foreign key).

Retrieving the Order

- To retrieve a complete order, data must be fetched from multiple tables (using JOINs).
- The rows are then converted back into an Order object with customer, list of line items and payment information.

Example SQL (to retrieve an order)

```
SELECT *
FROM ORDERS O
JOIN CUSTOMER C ON O.Customer_ID = C.Customer_I
JOIN ORDER_ITEM I ON O.Order_ID = I.Order_ID
JOIN PAYMENT P ON O.Order_ID = P.Order_ID
WHERE O.Order_ID = 1001;
```

Key Takeaways

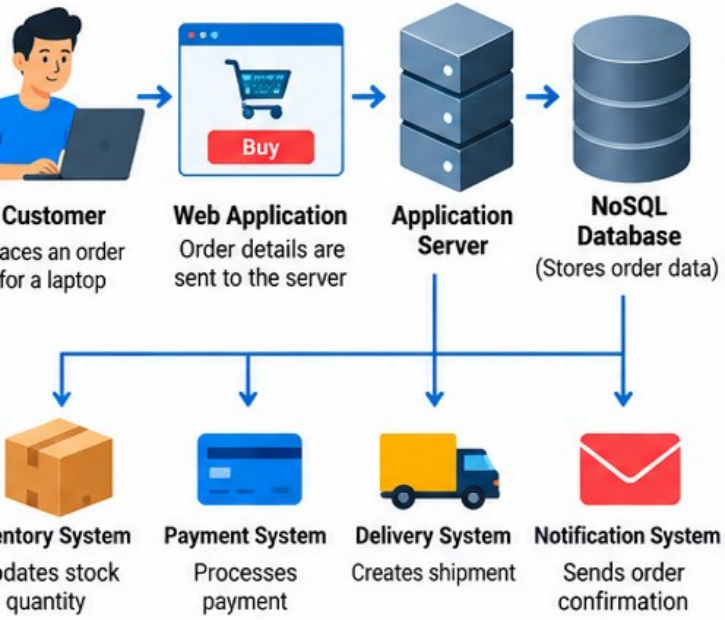
- Impedance mismatch occurs because the application uses an object-based structure, while the relational database uses a table-based structure.
- A single Order object is stored across multiple tables (CUSTOMER, ORDERS, ORDER_ITEM, PAYMENT).
- Data must be mapped between the two models, which adds development complexity.
- ORM frameworks (e.g., Hibernate, JPA, Entity Framework) can help manage this mapping.
- The example shows how nested objects and collections in the application are represented using related tables and foreign keys in a relational database.

Conclusion

The application and the relatio database represent the sam information in different way Converting between these tw representations is called impedance mismatch.

1. Integration in E-Commerce

n e-commerce application integrates multiple systems
o that a customer can place an order, make a payment,
date inventory, arrange delivery, and receive notifications.



Order Processing Flow

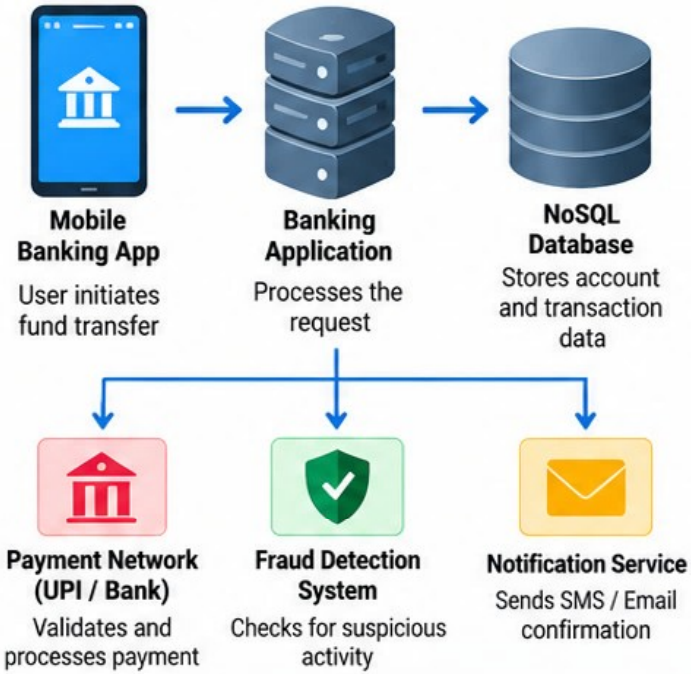
1. Customer places an order for a laptop.
2. Order information is stored in a NoSQL database.
3. Inventory system is updated (stock reduced).
4. Payment system processes the payment.
5. Delivery system creates a shipment.
6. Notification system sends a confirmation message.

esult

ifferent systems work together to complete the order
uccessfully. This is **integration.**

2. Integration in Banking / UPI

A banking application integrates with account systems, payment networks, fraud detection, and notification services to complete a money transfer.



Transaction Flow

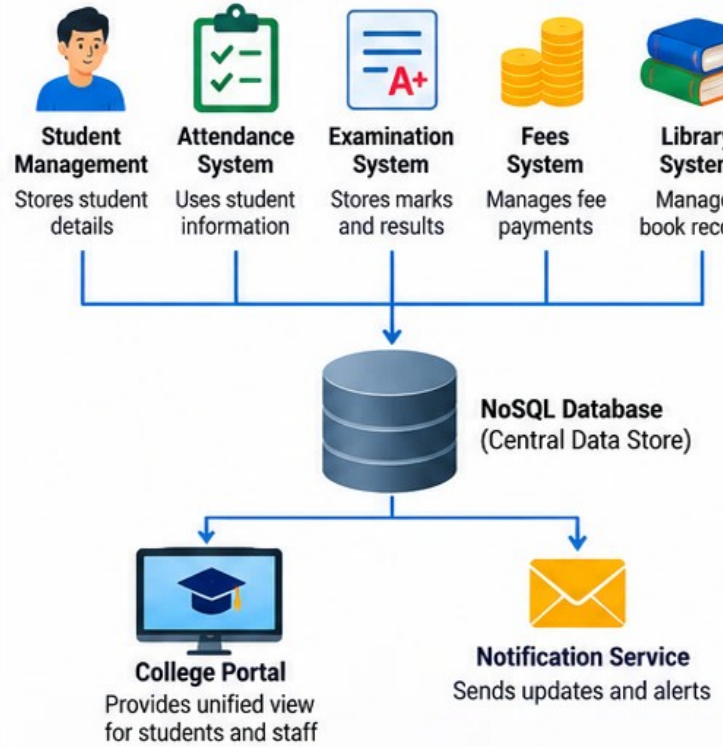
1. User enters from account, to account and amount.
2. Banking application retrieves account information.
3. Payment is processed (amount deducted and credited).
4. Transaction details are stored in the database.
5. Fraud detection system checks the transaction.
6. Notification service sends a confirmation message.

Result

Multiple systems work together to complete the transaction securely and efficiently. This is **integration.**

3. Integration in College Managemen

A college management system integrates student, attendar
examination, fees, library and notification systems so tha
information can be shared and used across departments



Example Use Case: Student Report

- Student registers in the student system.
- Attendance system uses the same student information.
- Examination system records marks.
- Fees system updates payment status.
- All information is available in the college portal.
- Student can view attendance, marks, fee status, and other detail
  in one place.

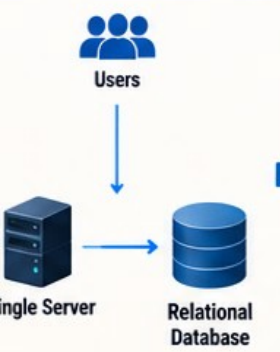
Result

Different systems are connected and share information.
This improves efficiency and provides a better experience for studer
This is integration.

1. Why NoSQL Emerged?
Changing Needs of Modern Applications

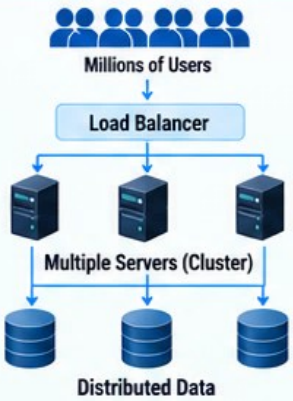
om Traditional to Modern Applications

rlier (Small-scale Applications)



- Few users
- Manageable amount of data
- Works well with relational databases

Today (Large-scale Web Applications)



- Huge number of users
- Very large and diverse data
- High number of requests
- Data stored across many machines

asons for the Shift

- Rapid growth in number of users
- Massive increase in data volume
- High request and transaction rates
- Need for distributed and clustered computing
- Non-uniform and rapidly changing data
- Limitations and high cost of scaling a single machine



- More Users
- More Data
- More Traffic
- More Machines
- NoSQL Movement

ale Up vs Scale Out

ale Up (Vertical Scaling)



- Make one machine more powerful
- Increase CPU, RAM, storage
- Simple to manage
- Becomes very expensive
- Physical limits
- Single point of failure

Bigger Server
(e.g., 64 cores, 512 GB RAM)

Scale Out (Horizontal Scaling)



- Add more machines
- Use multiple standard machines
- Form a cluster
- More flexible and cost-effective
- Better fault tolerance
- Can grow gradually

Cluster of Machines
(e.g., 10, 100 or 1000 servers)

2. Key Milestones in the Emergence of NoSQL
From Big Ideas to a Movement

1990s — Early use of the term "NoSQL"

- Carlo Strozzi used the name NoSQL for an open-source relational database.
- It did not use SQL and was accessed through shell scripts.
- This earlier project had no influence on the modern NoSQL movement.

Strozzi's NoSQL (1990s)

2000s — Growth of large web applications

- Rapid increase in users, data and traffic.
- Organizations started using clusters of machines.
- Traditional database approaches faced new challenges in distributed environments.

Large Web Applications

Early 2000s — Google BigTable

- Developed a distributed data-storage system to handle very large datasets.
- Designed to work across large numbers of machines.
- Introduced new ideas for distributed storage.

Google BigTable

Early 2000s — Amazon Dynamo

- Developed a distributed storage system for high availability.
- Designed to operate at large scale.
- Influenced many later NoSQL databases.

Amazon Dynamo

June 11, 2009 — The NoSQL Meetup (San Francisco)

- Organized by Johan Oskarsson.
- Developers working on alternative databases met to present and discuss their projects.
- The name "NoSQL" was suggested by Eric Evans.
- The word was selected because it was short, memorable and easy to use as a Twitter hashtag.

NoSQL Meetup (2009)

2009 and after — Many open-source NoSQL projects

- Projects such as Cassandra, MongoDB, CouchDB, HBase and others gained attention.
- Open-source development helped wide adoption.
- The term NoSQL became associated with a broader movement.

Open Source Community

2010s and beyond — NoSQL becomes a movement

- No single definition or list of databases.
- Different types of databases for different needs.
- Polyglot persistence became common.
- Relational databases are still important; NoSQL adds more choices.

Multiple Database Technologies

The emergence of NoSQL was **driven by real-world needs, practical solutions and community-driven innovation."**

3. Characteristics, Examples and Impact
What NoSQL Represents Today

Common Characteristics of NoSQL Databases

1. Generally do not use standard SQL
2. Designed for distributed and clustered environments
3. Flexible data models (schema flexibility)
4. Handle large volumes of data and high traffic
5. Often developed as open-source projects
6. Provide high availability and fault tolerance
7. Support horizontal scaling (add more nodes)
8. Useful for non-uniform and rapidly changing data

Important Points

- NoSQL does not mea SQL is bad.
- There is no single definition of NoSQL.
- NoSQL is better understood as a movement rather than one technology
- Relational databases are still highly useful
- The focus is on choos the right tool for the right problem.

Types of NoSQL Databases (with Examples)

Document Database



- Stores data as documents (e.g., JSON)
- Flexible structure
- Suitable for non-uniform data

Example: MongoDB

Key-Value Database



- Stores data as key-value pairs
- Simple and fast
- Useful for caching and session data

Example: Redis

Column-Family Database



- Stores data in columns and column families
- Designed for large scale
- Suitable for distributed environments

Example: Cassandra

Graph Databa



- Stores data as and relationshi
- Useful for high connected data
- Suitable for so networks and recommendati

Example: Neo4j

Real-World Examples

E-Commerce



- Large product catalog
- Millions of customers
- Orders, inventory, reviews
- Handles high traffic during sales

Social Media



- User profiles, posts
- Likes, comments, shares
- Large volumes of activity data
- Requires high availability

IoT / Sensor Data



- Different types of sensor
- Non-uniform data struct
- Very high write volumes
- Suitable for distributed storage

Impact of the NoSQL Movement

- ✓ Expanded the range of database choices available.
- ✓ Encouraged innovation in distributed data storage.
- ✓ Led to the development of many open-source technologies.
- ✓ Made it practical to handle large-scale, real-time, and non-uniform data.
- ✓ Popularized the concept of polyglot persistence (different databases for different nee
- ✓ Changed the way organizations think about data storage and application design.

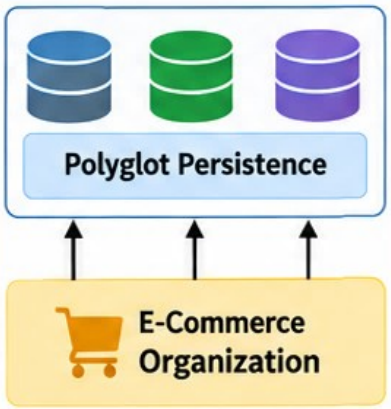
"NoSQL emerged to solve real problems created by the growth of modern applications and it continues to provide valuable options alongside relational databases."

1. What is Polyglot Persistence?

Polyglot Persistence means using different types of databases within the same organization, where each database is selected according to the requirements of a particular type of data or workload.

Why use it?

- Different data types have different requirements.
- One database may not be the best choice for everything.
- Each database is chosen for its strengths.
- Helps build scalable, efficient and maintainable systems.



Example Use Case

- Relational database for orders and payments (transactional data)
- Document database for product information (non-uniform data)
- Graph database for customer-product relationships (relationship-based data)

Use the right database for the right type of data.

2. Databases Used for Different Requirements

Relational Database (Transactional Data)

- Used for orders, payments, invoices and inventory.
- Provides ACID transactions and data integrity.
- Suitable for structured data and clear relationships.

Orders Table (Example)

Order_ID	Customer_ID	Amount	Status
O101	C001	50000	Placed
O102	C002	12000	Shipped
O103	C003	8000	Delivered

Relational DB
(e.g., MySQL, PostgreSQL, Oracle, SQL Server)

Document Database (Product Information)

- Used for product details, customer profiles, and other non-uniform data.
- Flexible schema allows different fields for different products.
- Suitable for frequently changing attributes.

Product Document (Examples)

```
{
  "name": "Laptop",
  "brand": "Dell",
  "RAM": "16 GB",
  "storage": "512 GB SSD",
  "processor": "Intel i7"
}
```

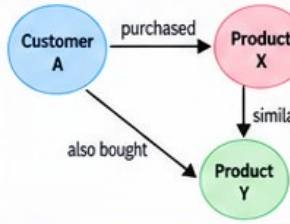
```
{
  "name": "Running Shoes",
  "brand": "Nike",
  "size": 9,
  "color": "Black",
  "material": "Leather"
}
```

Document DB
(e.g., MongoDB, CouchDB)

Graph Database (Relationship-Based Data)

- Used for customer-product relationships, recommendati and social connections.
- Stores data as nodes and edg
- Suitable for analyzing and traversing relationships.

Customer-Product Relationshi



Graph DB
(e.g., Neo4j, Amazon Nept

3. How It Works – Example: Customer Purchase Flow

onsider a customer purchasing a smartphone from the e-commerce platform.

Get Product Information (from Document DB)

application retrieves duct details from the ument database.

```
"product_id": "P101",
"name": "Smartphone",
"price": 25000,
"camera": "50 MP",
"battery": "5000 mAh"
```

Document DB

Product details with flexible attributes

2 Place Order and Process Payment (from Relational DB)

The order, payment and invoice are recorded in the relational database.

Order Table

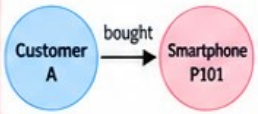
Order_ID	Cust_ID	Product_ID	Amount
O201	C001	P101	25000

Relational DB

Orders, payments, invoices (ACID transactions)

3 Update Relationships (in Graph DB)

The purchase relationship is recorded in the graph database.



Graph DB

Customer-product relationships

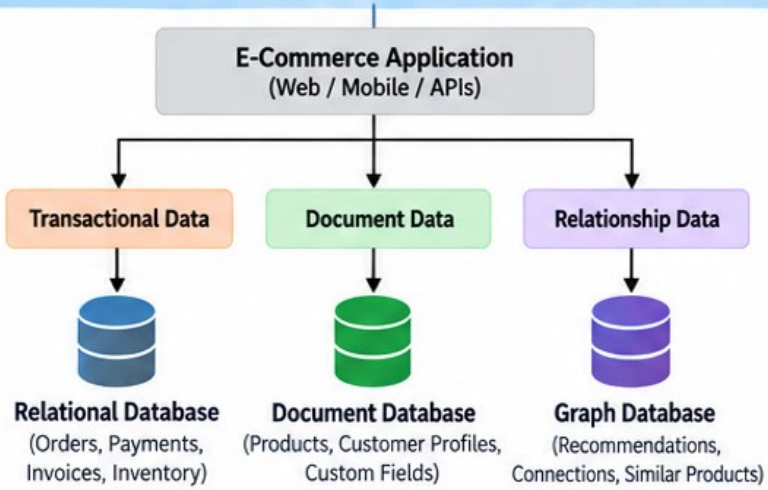
4 Generate Recommendations (from Graph DB)

Based on purchase relationships, the system finds similar products and recommends them.

Recommended:

- Mobile Cover
- Wireless Earbuds
- Screen Protector

4. Overall Architecture and Benefits



Key Benefits

- Uses the best database for e type of data.
- Meets different application requirements.
- Improves scalability and performance.
- Allows flexible data models.
- Supports complex relationsh analysis.
- More maintainable, efficient and future-ready systems.

Polyglot Persistence enables a fit-for-purpose approach in which each type of data is stored in the database that is most suitable for its characteristics and workload.

1. Scaling Up (Vertical Scaling)

caling Up means increasing the resources of a single server to handle more users nd data.

Before (Lower Capacity)



CPU : 4 cores
RAM : 16 GB
Storage : 500 GB

Single Server

Upgrade Resources

After (Higher Capacity)



CPU : 16 cores
RAM : 64 GB
Storage : 2 TB

Single Server (more powerful)

Advantages

- Simple to implement
- Fewer software changes
- Easier to manage
- Useful for moderate growth

Disadvantages

- Limited by single server capacity
- Can be expensive
- Single point of failure
- May require downtime
- Not sufficient for rapid growth

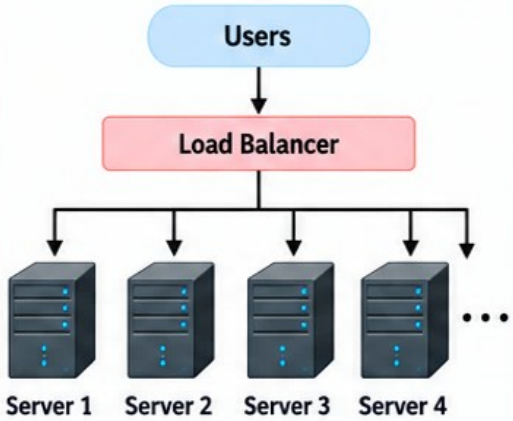
2. Scaling Out (Horizontal Scaling)

Scaling Out means adding more servers and distributing the workload among them

Initial Setup



After Scaling Out



Advantages

- Handles large number of us
- Easy incremental growth
- Better fault tolerance
- Higher availability
- Distributes workload
- Suitable for large-scale applications

Disadvantages

- More complex to desig and manage
- Requires load balancin
- Data synchronization can be challenging
- Distributed failures are more difficult to hand

3. Applying to a Growing Web Application

xample: An e-commerce application experiencing rapid growth.

Current Situation

- 1 million users
- 10 TB of data
- Thousands of requests per second

Rapid Growth

After Some Time

- 10 million users
- 100 TB of data
- Much higher request volume

Using Scaling Up

Upgrade the existing server to a more powerful machine.

Improves capacity, but eventually reaches the maximum limit of a single server.

Suitable only for moderate growth.
May not be sufficient for very large growth.

Using Scaling Out

- Add multiple web servers and use a load balancer.
- Distribute the application workload across servers.
- Use multiple database nodes (sharding/partitioning) to handle large data volume.

More suitable for rapid growth in users and data volume.

4. Comparison and Final Recommendation

Feature	Scaling Up (Vertical)	Scaling Out (Horizontal)
Meaning	Increase resources of one server	Add more servers
Number of servers	Usually one	Multiple
Maximum capacity	Limited by one machine	Can grow by adding machines
Implementation	Relatively simple	More complex
Fault tolerance	Lower	Higher
Load distribution	Limited	Excellent
Hardware upgrade	Required	Additional servers are added
Large-scale suitability	Limited	Very suitable
Rapid growth	Less suitable	More suitable

Conclusion

For a web application whose users and data volume are increasing rapidly, Scaling Out is the more suitable approach.

- It provides better scalability, availability and fault tolerance.
- It allows the application to handle a much larger workload.
- Servers and database nodes can be added incrementally as demand increases.
- In practice, organizations may use a combination of both approaches (scale up individual servers and scale out b adding more servers).

1. What is Scaling Up (Vertical Scaling)?

Scaling Up means increasing the resources of a single server.
We make the existing server more powerful by adding CPU, RAM, storage or network capacity.

Before (Lower Capacity)



Single Server

CPU: 4 cores
RAM: 16 GB
Storage: 500 GB
Network: 1 Gbps

Upgrade Resources

After (Higher Capacity)



Single Server (More Powerful)

CPU: 16 cores
RAM: 64 GB
Storage: 2 TB
Network: 10 Gbps

The application continues to run on one server, but the server has more resources.

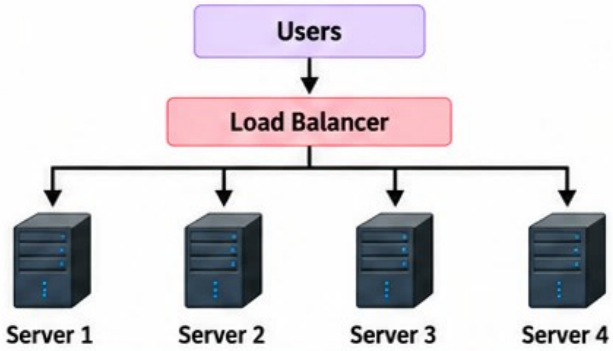
2. What is Scaling Out (Horizontal Scaling)?

Scaling Out means adding more servers to distribute the workload.
Instead of making one server more powerful, we use multiple servers.

Initial Setup



After Scaling Out



The load balancer distributes incoming requests among multiple servers.
More servers can be added as demand increases.

3. Advantages and Disadvantages

Scaling Up (Vertical Scaling)

Advantages

- Simple to implement.
- Usually requires fewer software changes.
- Easier to manage (single server).
- Useful for smaller or moderately growing systems.

Disadvantages

- Limited by maximum server capacity.
- High-end hardware can be expensive.
- Single point of failure.
- Upgrading hardware may require downtime.
- Not sufficient for very large applications.

Scaling Out (Horizontal Scaling)

Advantages

- Can handle a very large number of users.
- Additional servers can be added as needed.
- Better fault tolerance.
- Workload can be distributed.
- Provides higher availability.
- Works well with cloud-based and distributed systems.

Disadvantages

- More complex to design and manage.
- Requires communication between servers.
- Data synchronization can be challenging.
- Needs load balancing.
- Distributed failures are more difficult to handle.

4. Key Differences and Real-World Example

Feature	Scaling Up (Vertical)	Scaling Out (Horizontal)
Basic idea	Make one server more powerful	Add more servers
Number of servers	Usually one	Multiple
CPU/RAM	Increase in existing server	Distributed across servers
Complexity	Relatively simple	More complex
Hardware limit	Limited by server capacity	Can continue adding servers
Fault tolerance	Generally lower	Generally higher
Load distribution	Limited	Workload can be distributed
Cost	Powerful hardware can be expensive	Cost grows as servers are added
Common use	Smaller or moderately growing systems	Large-scale applications

Real-World Example: Online Shopping Application

Scaling Up



Upgrade the existing server:

- From 8 CPU cores, 32 GB RAM
- To 32 CPU cores, 128 GB RAM

Same server, higher capacity.

Scaling Out



Add more servers:

Start with 1 server, then 2, 4, ... more to handle growing users.

For large-scale applications, scaling out is often preferred because it provides greater scalability, availability and fault tolerance.

. What are NoSQL Databases?

oSQL databases are databases designed to store and manage large amounts f data, especially when the data is **unstructured, semi-structured, apidly changing**, or **distributed** across many computers.

ey Points

Do not use traditional table-based structure.
Provide flexible data models.
Designed for scalability and high performance.
Well-suited for modern web applications.
Often open-source with active communities.
Used when data is large, diverse, and frequently changing.

oSQL is an alternative approach to data storage, built for the needs of modern applications."

Traditional vs NoSQL

Relational Database (Traditional)

Student

ID	Name	Age
1	Ravi	20
2	Sita	21
3	Kiran	22

Uses tables with fixed schema (all rows have the same structure).

NoSQL Database (Example: Document)

```
{
  "id": 1,
  "name": "Ravi",
  "age": 20,
  "course": "CSE"
}
```

```
{
  "id": 2,
  "name": "Sita",
  "age": 21,
  "course": "ECE",
  "skills": ["Python","ML"]
}
```

Stores data in flexible format (different records can have different fields).

2. Types of NoSQL Databases

NoSQL databases use different data models to store and access data.
The main types are:

1. Document Database

- Stores data as documents (e.g., JSON).
- Flexible schema.
- Suitable for non-uniform data.

```
{
  "name": "Ravi",
  "age": 20,
  "department": "CSE",
  "ckills": ["Python","DBMS"]
}
```

Example: MongoDB

2. Key-Value Database

- Stores data as key-value pairs.
- Very fast and simple.
- Useful for caching and session data.

Key	Value
user:1	Ravi
user:2	Sita
session:1	XYZ123
cart:1	["item1","item

Example: Redis

3. Column-Family Database

- Stores data in columns and column families.
- Designed for large-scale data.
- Suitable for distributed environments.

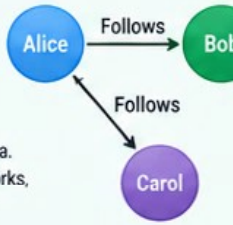
User (Column Family)

id	name	email	location
1	Ravi	r@example.com	IN
2	Sita	s@example.com	IN

Example: Cassandra

4. Graph Database

- Stores data as nodes and relationships.
- Useful for highly connected data.
- Commonly used in social networks, recommendation systems, etc.



Example: Neo4j

. Key Features of NoSQL Databases

Flexible Schema
elds can be addied or modified asily without changing the ntire structure.

Horizontal Scaling
Capacity can be increased by adding more machines (nodes).

Distributed Storage
Data can be stored across multiple nodes in a cluster.

High Availability
Systems can continue operating even if some nodes fail.

High Performance
Designed to handle large volumes of data and high request rates.

Open-Source Ecosystem
Many NoSQL systems are open-source with active communities.

Key Idea:
NoSQL databases are built to be **scalable, flexible,** and **reliable** for modern, large-scale applications.

4. Real-World Example and Use Cases

Example: E-Commerce Application



Sample Product Document
(in a document database)

```
{
  "product_id": "P1001",
  "name": "Laptop",
  "price": 59999,
  "brand": "Dell",
  "specs": {
    "ram": "16GB",
    "storage": "512GB",
    "processor": "Intel i5"
  },
  "in_stock": true
}
```

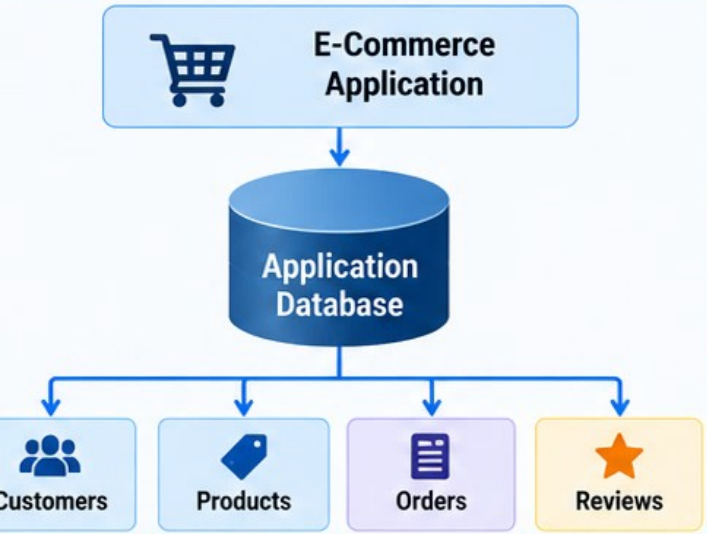
Other Use Cases

- **Social Media** (user profiles, posts, likes)
- **IoT and Sensor Data** (large, non-uniform data)
- **Mapping Applications** (geographic data, routes)
- **Media & Content Platform** (videos, images, metadata
- **Retail and E-Commerce** (product catalog, orders)
- **Real-Time Analytics** (logs, events, activity data

Summary:
NoSQL databases provide flexible and scalable solutions for large, distributed, and rapidly changing data, making them ta valuable choice for many modern applications.

1. Application Database

n application database is a database that is designed nd used primarily by **one particular application**. stores the data that the application needs to perform s own operations.



xample: Customer Data in E-Commerce

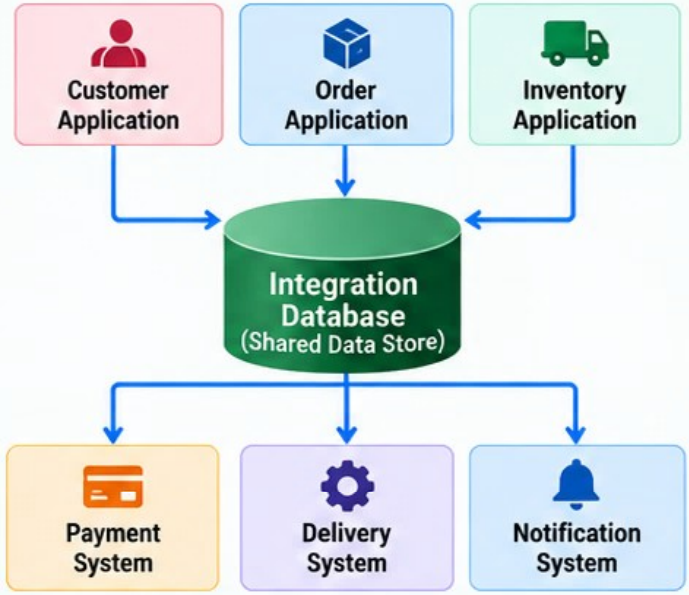
```
  "customer_id": 101,
  "name": "Ravi",
  "email": "ravi@example.com",
  "address": {
    "city": "Guntur",
    "state": "Andhra Pradesh"
  }
```

ey Characteristics

- Supports a specific application
- Stores application-specific data
- Accessed mainly by that application
- Designed according to application requirements
- Can be optimized for the application's workload
- Can use SQL or NoSQL depending on requirements

2. Integration Database

An integration database is a shared data store that helps **different applications or systems** exchange and access common information. It acts as a central point for data exchange.



Example: Shared Order Data

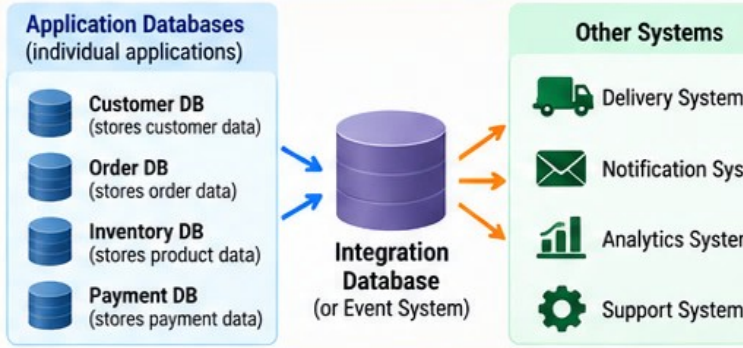
```
{
    "order_id": "ORD1001",
    "customer_id": 101,
    "product_id": "P500",
    "quantity": 1,
    "amount": 50000,
    "status": "PAYMENT_PENDING"
}
```

Key Characteristics

- Used for data exchange among multiple applications
- Stores shared or common data
- Supports integration and interoperability
- Designed based on integration requirements
- Can be accessed by multiple systems
- Often implemented using APIs, events, or message brokers (rather than a single shared database)

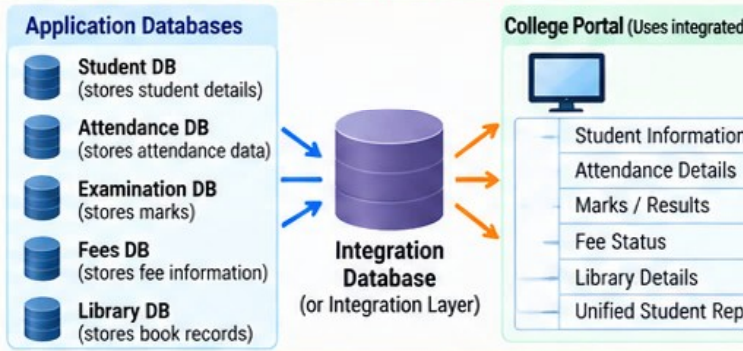
3. Real-World Examples

Example 1: E-Commerce System



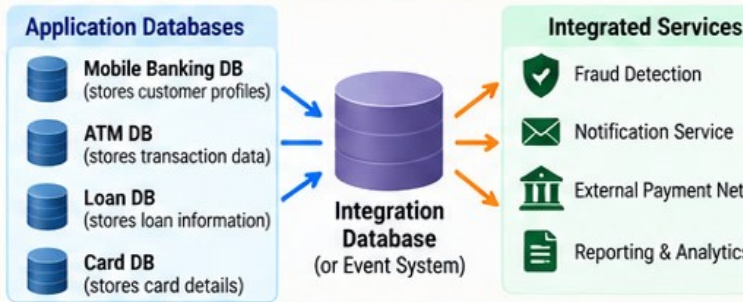
When a customer places an order, the order information is shared with inventor payment, delivery and notification systems through the integration database.

Example 2: College Management System



The college portal collects information from different systems through integratio and provides a unified view to students and staff.

Example 3: Banking System



When a transaction is performed, multiple services exchange information like authentication, account details, fraud check, and notifications through integration

1. What is "Attack of the Clusters"?

Attack of the Clusters" refers to the rapid growth and doption of clusters (multiple machines working together) handle the huge increase in data, users, and traffic in large eb applications. It is **not a security attack.** It describes ow the need for large-scale applications pushed rganizations to use distributed clusters instead of a single rge machine.

When a single machine can no longer handle the load, we add more machines and form a cluster.

hy Clusters Became Necessary?

Rapid growth in number of users
Massive amounts of data
High number of requests
Social, web, and multimedia applications
Large-scale data sush as logs, maps, links, and user interactions
Single machines became expensive and had physical limits

caling Options

Scale Up (Vertical Scaling)

Make one machine more powerful
Increase CPU, RAM, storage
Becomes very expensive
Has physical limits
Single point of failure



Bigger Server
(e.g., 64 cores, 512 GB RAM)

2. Scale Out (Horizontal Scaling)

- Add more machines
- Use multiple smaller/standard machines
- Form a cluster
- More flexible and cost-effective
- Better fault tolerance



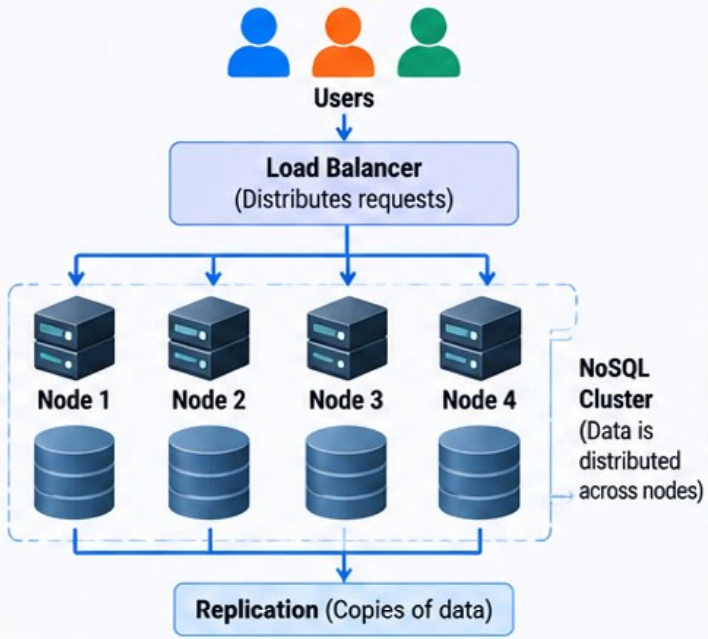
Cluster of Machines
(Scale Out)

ey Takeaway

e rapid growth of web applications led to the "attack" (rise) of clusters, hich created the environment for NoSQL databases.

2. How Clusters Work in NoSQL

Basic Architecture of a Cluster



Benefits of Clusters

- Handles large number of users and requests
- Stores very large amounts of data
- Provides high availability (continues working even if a node fails)
- Supports horizontal scaling (add more nodes)
- Uses commodity hardware (standard machines)
- Common in many NoSQL databases (e.g., MongoDB, Cassandra, HBase)

Challenges Introduced by Clusters

- Network communication between nodes
- Deciding how to distribute data (sharding/partitioning)
- Keeping multiple copies of data (replication)
- Managing consistency across nodes
- Handling node failures
- Monitoring and cluster management

Clusters solve big data and high traffic problems, but they also introduce new challenges.

3. Real-Time Examples

Example 1: E-Commerce Website

Scenario

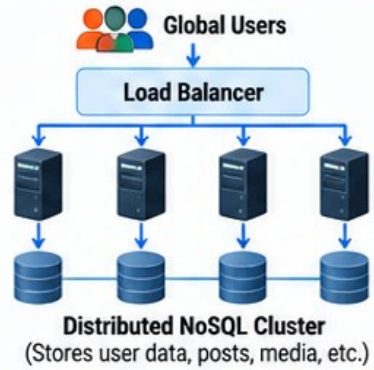
- During a festival sale, millions of customers visit the website.
- Product searches, cart updates, orders and payments create huge traffic.
- A single server cannot handle the load.



Example 2: Social Media Platform

Scenario

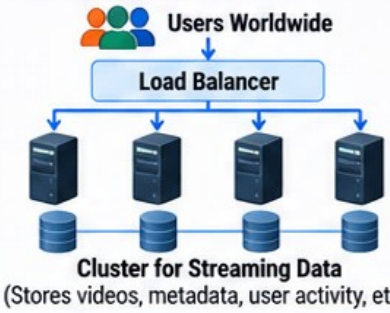
- Millions of users create posts, comments, likes, shares, and messages.
- Generates huge amounts of data every day.
- Needs to serve users worldwide with high availability.



Example 3: Video Streaming Platform

Scenario

- Millions of users stream videos simultaneously.
- Stores large video files, viewing history, and recommendations.
- Needs high bandwidth and global distribution.



Overall Message

Large-scale applications in e-commerce, social media, and video streami need clusters to handle massive users, data, and requests. This is the esse of the "Attack of the Clusters" and a key reason for the adoption of NoSQ

Concurrency in Databases

Concurrency means **multiple users or processes** can access and work with the **same data** at the same time.
A database uses concurrency control (like locks, timestamps, or optimistic control) to keep data consistent and avoid conflicts.



Multiple users accessing data simultaneousl

Example 1: Online Banking

Two people try to withdraw money from the same account at the same time.

Initial Balance: ₹ 10,000



Database (with concurrency control)

Processes both transactions safely.
Result: One transaction succeeds and the other is handled correctly (e.g., allowed if balance is enough or rejected).

Outcome:

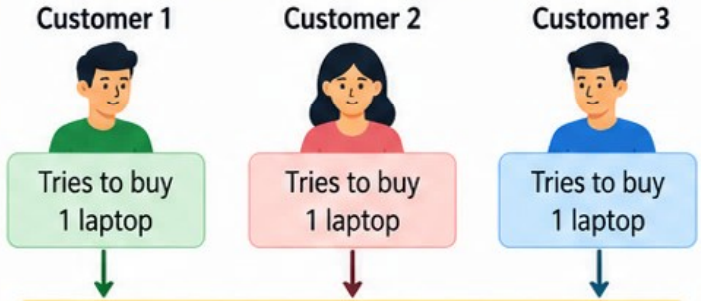
The account balance remains correct, and no money is lost or double-withdrawn.

Example 2: E-commerce (Inventory)

Many customers try to buy the same product at the same time.

Product: Laptop
Available Quantity: 5





Database (handles concurrent requests)

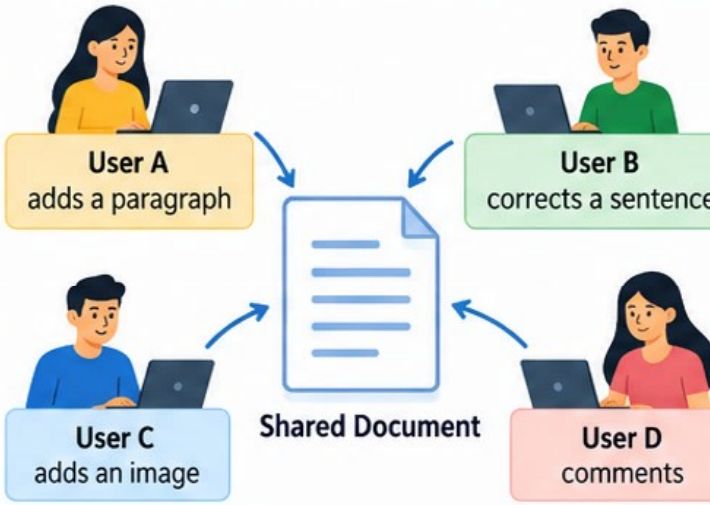
Updates inventory safely so that no more than 5 laptops are sold.

Outcome:

Only 5 customers can successfully purchase the laptop. Others are informed that it is out of stock.

Example 3: Google Docs (Collaborative Editin

Multiple users edit the same document at the same tim



Database (supports concurrent updates)

Merges all changes and keeps the document consistent in real time.

Outcome:

All users can work on the same document simultaneously, and everyone's changes are saved correctly.

Concurrency allows many users to work with the same data at the same time, while the database ensures correct, consistent, and reliable results.